

R-Control IP4 – HTTP JSON REST API ("/api/... ")

Diese Dokumentation beschreibt die HTTP/HTTPS JSON REST API des R-Control IP4. Alle Angaben sind aus der Firmware-Implementierung abgeleitet und beziehen sich auf die aktuell im Projekt enthaltenen Endpunkte.

1. Transport, Base-URL und Ports

- **Base-URL:** `http(s)://<gerät>:<port>`
- **HTTP-Port:** `config.network.lan.eth0.http.port` (Default: `80`)
- **HTTPS** (optional): `config.network.lan.eth0.https.enabled`
 - Zertifikat/Key werden aus SPIFFS geladen:
 - `config.network.lan.eth0.https.cert_path` (Default: `/spiffs/server.crt`)
 - `config.network.lan.eth0.https.key_path` (Default: `/spiffs/server.key`)
 - Wenn HTTPS aktiviert ist, aber Zertifikat/Key nicht lesbar sind, fällt das Gerät automatisch auf HTTP zurück.

Hinweis: Die Firmware unterstützt sowohl HTTP als auch HTTPS, abhängig von der Build-Konfiguration (ESP-IDF HTTPS-Server Feature) und der Runtime-Konfiguration.

2. Authentifizierung

2.1 Bearer-Token

Alle Endpunkte unter `/api/...` (außer Login) erfordern ein gültiges Bearer-Token:

Authorization: `Bearer <token>`

Wenn der Header fehlt/zu lang ist oder der Token ungültig ist:

- **HTTP 401 Unauthorized**
- Header: `WWW-Authenticate: Bearer realm="R-Control-IP4"`
- Body:

```
{"error": "unauthorized"}
```

2.2 Login – `POST /api/auth/login`

Request

- Methode: `POST`
- Pfad: `/api/auth/login`
- Body: JSON (max. 1024 Bytes)

```
{  
  "username": "admin",
```

```
"password": "<password>",
"ttl": 3600
}
```

- `ttl` ist optional und wird auf **60...86400 Sekunden** begrenzt.

Response (200 OK)

```
{
  "token": "<jwt>",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "name": "admin",
    "username": "admin",
    "permissions": ["settings", "overview", "about"]
  }
}
```

Fehlerfälle

- **401 Unauthorized** mit `{"error": "invalid_credentials"}` (falsche Zugangsdaten)
- **400 Bad Request** (ungültige Länge/JSON; hierbei kann die Firmware auch Text-Fehlerantworten verwenden)

Beispiel (curl)

```
curl -sS -X POST "http://r-control-ip4.local/api/auth/login" \
-H "Content-Type: application/json" \
-d '{"username": "admin", "password": "admin123", "ttl": 3600}'
```

3. Berechtigungen (Permissions)

Einige Endpunkte verlangen zusätzlich zur Authentifizierung eine Berechtigung im Benutzerprofil:

- **settings**: Konfiguration, Benutzerverwaltung, Neustart/OTA, Factory Provisioning
- **overview**: Telemetrie/Übersicht (Temperatur-Historie, IO-Log)
- **about**: Firmware-/Factory-Info lesen (teilweise alternativ zu **settings**)

Wichtig: Mehrere Endpunkte prüfen **nur Authentifizierung** (keine Permission). Das ist absichtlich so dokumentiert, weil es dem aktuellen Verhalten entspricht.

4. Antwortformate und Fehlerhandling

4.1 Erfolgsantworten

- Erfolgsantworten sind i. d. R. JSON.

- Viele Antworten setzen Cache-Busting (`Cache-Control: no-store, no-cache, must-revalidate`).

4.2 Standard-JSON-Fehler

Ein Großteil der validierenden/permission-bezogenen Fehler nutzt:

```
{"error": "<code>", "message": "<beschreibung>"}
```

Beispiele:

- 403 Forbidden: `{"error": "forbidden", "message": "Berechtigung 'settings' erforderlich"}`
- 400 Bad Request: `{"error": "invalid_json", "message": "Ungültiges JSON"}`

4.3 Text-Fehlerantworten

Einige Handler nutzen ESP-IDF `httpd_resp_send_err(...)` und liefern **Text** (nicht JSON). Clients sollten daher robust sowohl JSON als auch Textfehler verarbeiten.

5. Endpunkte

5.1 Netzwerk

GET /api/network

- Auth: erforderlich
- Permission: keine (auth-only)
- Response: `config.network` als JSON.

GET /api/network/status

- Auth: erforderlich
- Permission: keine (auth-only)
- Response: Live-Status (Beispielstruktur, Felder können leer sein):

```
{
  "enabled": true,
  "protocol": "dhcp+fallback",
  "netifReady": true,
  "ip": "192.168.0.3",
  "netmask": "255.255.255.0",
  "gateway": "192.168.0.1",
  "ipAssigned": true,
  "mac": "AA:BB:CC:DD:EE:FF",
  "linkUp": true,
  "online": true,
  "dhcpActive": true,
  "dhcpStatus": "started",
  "dns": ["192.168.0.1"],
```

```
"source": "dhcp",
"internetReachable": true,
"wanIpValid": true,
"wanFetching": false,
"wanCheckedMs": 0,
"wanIp": "203.0.113.10",
"wanGeoValid": false
}
```

POST /api/network/wan-refresh

- Auth: erforderlich
- Permission: keine (auth-only)
- Startet ein WAN/Geo-Refresh im Hintergrund.
- Response:
 - {"status": "started"} oder {"status": "busy"}

POST /api/network/geo/address

- Auth: erforderlich
- Permission: settings
- Body: JSON (max. 1024 Bytes)
 - Pflicht: street, houseNumber, city, postalCode
 - Optional: country
- Response: status: ok + Geodaten.
- Typische Fehler:
 - 400 missing_fields
 - 404 geo_not_found
 - 400 geo_invalid
 - 502 geo_failed

POST /api/network/eth0

- Auth: erforderlich
- Permission: keine (auth-only)
- Body: JSON (max. 2048 Bytes) – erlaubt sind String-Keys:
 - protocol, ip, netmask, gateway, dns, hostname
- Response:

```
{"status": "updated"}
```

POST /api/network/apply

- Auth: erforderlich
- Permission: keine (auth-only)
- Body: keiner

- Response:

```
{"status": "restarting"}
```

- Verhalten: Neustart wird nach der Antwort zeitverzögert ausgelöst.

5.2 System

GET /api/system

- Auth: erforderlich
- Permission: keine (auth-only)
- Response: `config.system` als JSON.

POST /api/system/restart

- Auth: erforderlich
- Permission: `settings`
- Response: `{"status": "restarting"}`
- Verhalten: Neustart wird nach der Antwort zeitverzögert ausgelöst.

POST /api/system/ota

- Auth: erforderlich
- Permission: `settings`
- Body: **binäres Firmware-Image** (kein JSON)
- Limits:
 - Payload darf nicht leer sein
 - Payload muss in die OTA-Partition passen (sonst `413 image_too_large`)
- Response (Beispiel):

```
{
  "status": "ok",
  "written": 1048576,
  "partition": "ota_0",
  "reboot": true,
  "firmware": {
    "version": "1.2.3",
    "project": "R-Control-IP4",
    "build_date": "Jan 01 2026",
    "build_time": "12:34:56"
  }
}
```

- Verhalten: Neustart wird nach der Antwort zeitverzögert ausgelöst.

5.3 Online-Firmware-Update (GitHub)

Diese Endpunkte nutzen `config.system.firmwareUpdate.*` (z. B. `enabled`, `owner`, `repo`, `asset`).

GET /api/system/fw-update/check

- Auth: erforderlich
- Permission: `settings`
- Response (Beispiel):

```
{
  "status": "ok",
  "currentVersion": "1.0.0",
  "latestVersion": "1.1.0",
  "updateAvailable": true,
  "assetName": "R-Control-IP4.bin",
  "assetUrl": "https://...",
  "prerelease": false,
  "draft": false
}
```

POST /api/system/fw-update/install

- Auth: erforderlich
- Permission: `settings`
- Startet das Online-Update im Hintergrund.
- Response: `202 Accepted` mit `{"status": "started", ...}`
- Fehler: `409 fw_update_busy`, `400 fw_update_disabled`, `400 fw_update_not_configured`, `502 fw_update_check_failed`

GET /api/system/fw-update/status

- Auth: erforderlich
- Permission: keine (auth-only)
- Response:

```
{"status": "ok", "state": "idle", "message": "...", "error": ""}
```

5.4 Scheduler

GET /api/scheduler

- Auth: erforderlich
- Permission: keine (auth-only)
- Response: Scheduler-Konfiguration (JSON), zusätzlich:
 - `runtime_rule_count`
 - `runtime_max_rules`

POST /api/scheduler

- Auth: erforderlich
- Permission: keine (auth-only)
- Body: JSON-Objekt (max. 64 KiB)
- Besonderheit:
 - Wenn das Payload `astro` oder `location` enthält, werden diese Daten in `config.system.astro` persistiert.
- Response: `status: ok|error` und `data` (aktuelle Scheduler-Konfig mit Runtime-Metadaten).

5.5 Actions

GET /api/actions

- Auth: erforderlich
- Permission: keine (auth-only)
- Response: Actions-Konfiguration (JSON) + Runtime-Metadaten analog Scheduler.

POST /api/actions

- Auth: erforderlich
- Permission: keine (auth-only)
- Body: JSON-Objekt (max. 64 KiB)
- Response: `status: ok|error` und `data` (aktuelle Actions-Konfig mit Runtime-Metadaten).

5.6 Konfiguration

GET /api/config

- Auth: erforderlich
- Permission: keine (auth-only)
- Response: komplette aktive Konfiguration als JSON (Streaming), **ohne** `config.auth.jwt_secret`.

POST /api/config

- Auth: erforderlich
- Permission: keine (auth-only)
- Body: JSON (max. 64 KiB)
- Verhalten:
 - Falls `config.auth.jwt_secret` bereits gesetzt ist, wird es beim Ersetzen der Konfiguration beibehalten.
- Response: `{"status":"ok"}`

GET /api/config/bundle

- Auth: erforderlich
- Permission: `settings`
- Response: `application/x-tar` Download, enthält:

- `config.json`
- `scheduler.json`
- `actions.json`

Wichtiger Hinweis: Dieses Bundle enthält die aktuelle Konfiguration, einschließlich sensibler Felder (z. B. `jwt_secret`). Behandeln Sie den Export wie ein Passwort-Backup.

POST /api/config/bundle

- Auth: erforderlich
- Permission: `settings`
- Body: `application/x-tar` Upload (max. ca. 768 KiB)
- Erwartete Inhalte: `config.json`, `scheduler.json`, `actions.json`
- Response: `{"status": "imported", "reboot": true}`
- Verhalten: Neustart wird nach der Antwort zeitverzögert ausgelöst.

POST /api/config/restore-defaults

- Auth: erforderlich
- Permission: `settings`
- Response: `{"status": "restored"}`
- Verhalten: Werkseinstellungen werden geladen und das Gerät startet danach neu.

5.7 Telemetrie / Übersicht

GET /api/telemetry/temperature

- Auth: erforderlich
- Permission: `overview`
- Response: Streaming JSON
 - `intervalSeconds`
 - `capacity`
 - `minIntervalSeconds`, `maxIntervalSeconds`
 - `samples`: Array aus `{ "t": <timestampMs>, "v": <temperatureC> }`

GET /api/overview/actions

- Auth: erforderlich
- Permission: `overview`
- Response: `entries`-Array aus IO-/Relaisereignissen inkl. Klartextfeldern.

5.8 About

GET /api/about/app

- Auth: erforderlich
- Permission: `settings` **oder** `about`
- Response:

```
{"status": "ok", "appVersion": "...", "idfVersion": "..."}"
```

5.9 Factory (Fertigung)

Diese Endpunkte sind für Provisionierung/Produktion gedacht.

GET /api/factory/info

- Auth: erforderlich
- Permission: **settings** **oder** **about**
- Response enthält u. a. **articleNumber**, **serialNumber**, **productionDate**, **macAddress**, **factoryMac**, **locked**, **ntcCalibration**.

POST /api/factory/provision

- Auth: erforderlich
- Permission: **settings**
- Body: JSON (max. 1024 Bytes)
 - Pflicht: **articleNumber**, **serialNumber**, **productionDate** (YYYY-MM-DD)
 - Optional: **macAddress** (setzt Base-MAC)
 - Optional: **calibrateNtc** (bool), **ntcRefTempC** (number)
- Fehler:
 - **409** **factory_locked**
 - **409** **already_provisioned**
 - **400** **invalid_date**, **400** **invalid_mac**, **400** **invalid_chars**

POST /api/factory/lock

- Auth: erforderlich
- Permission: **settings**
- Sperrt den Factory-Bereich (wenn Provisionierung vollständig ist).

5.10 Benutzerverwaltung

Alle User-Endpoints erfordern **settings**.

GET /api/users

- Response:

```
{"limit": 8, "users": [ ... ]}
```

POST /api/users

- Body: JSON (max. 4096 Bytes)

- Pflicht: `username`, `password`
- Optional: `permissions` (Array)
- Response: `201 Created` mit `{"status": "created"}`

PUT `/api/users/<username>`

- Body: JSON (max. 4096 Bytes)
 - Optional: `permissions` (Array)
 - Optional: `password` (String, erfordert zusätzlich `currentPassword`)
 - Optional: `currentPassword`
- Response: `{"status": "updated"}`

DELETE `/api/users/<username>`

- Response: `{"status": "deleted"}`
- Fehler: `409 last_user` (letzter Benutzer kann nicht entfernt werden)

6. Best Practices & Security

- Nutzen Sie **HTTPS**, sobald das Gerät außerhalb eines vollständig vertrauenswürdigen Netzes betrieben wird.
- Behandeln Sie Tokens und Konfig-Exporte (insbesondere `/api/config/bundle`) als **geheim**.
- Schreiben/Reset/OTA-Funktionen sollten nur Konten mit `settings` erhalten.
- Beachten Sie, dass einige Endpunkte derzeit nur Authentifizierung prüfen (keine Permission). Wenn Sie ein restriktiveres Modell benötigen, muss dies in der Firmware umgesetzt werden.

7. Troubleshooting

- `401 Unauthorized`: Token fehlt/ungültig/abgelaufen oder `Authorization` Header zu lang.
- `403 Forbidden`: Benutzer hat nicht die benötigte Permission.
- `400 invalid_json`: JSON ist syntaktisch ungültig.
- Nach `.../apply`, `.../restore-defaults`, `.../ota` oder Bundle-Import ist ein **Neustart** Teil des normalen Ablaufs.